Introduction to Machine Learning: Case-based Reasoning & Unsupervised Learning

Maciej Piasecki

Similarities as a Basis

- Class members should be similar in some sense (Kubat, 2017)
  - „similar objects often belong to the same class"
- Case-based Reasoning
  - other names: Memory-based learning, Instance-based learning
  - general idea
    - we have a set of objects of known classes
    - „to determine the class of object x, find the training example most similar to it." (Kubat, 2017)
    - „Then label x with this example's class."
- Key questions:
  - how to assess similarity of objects?
  - how similar should a training object be to make a good decision?
  - Is one similar training object enough to make a good decision?
- Simplifying abstraction
  - objects represented by attributes
  - similarity as a function of attribute-value vectors

Similarities of objects – example

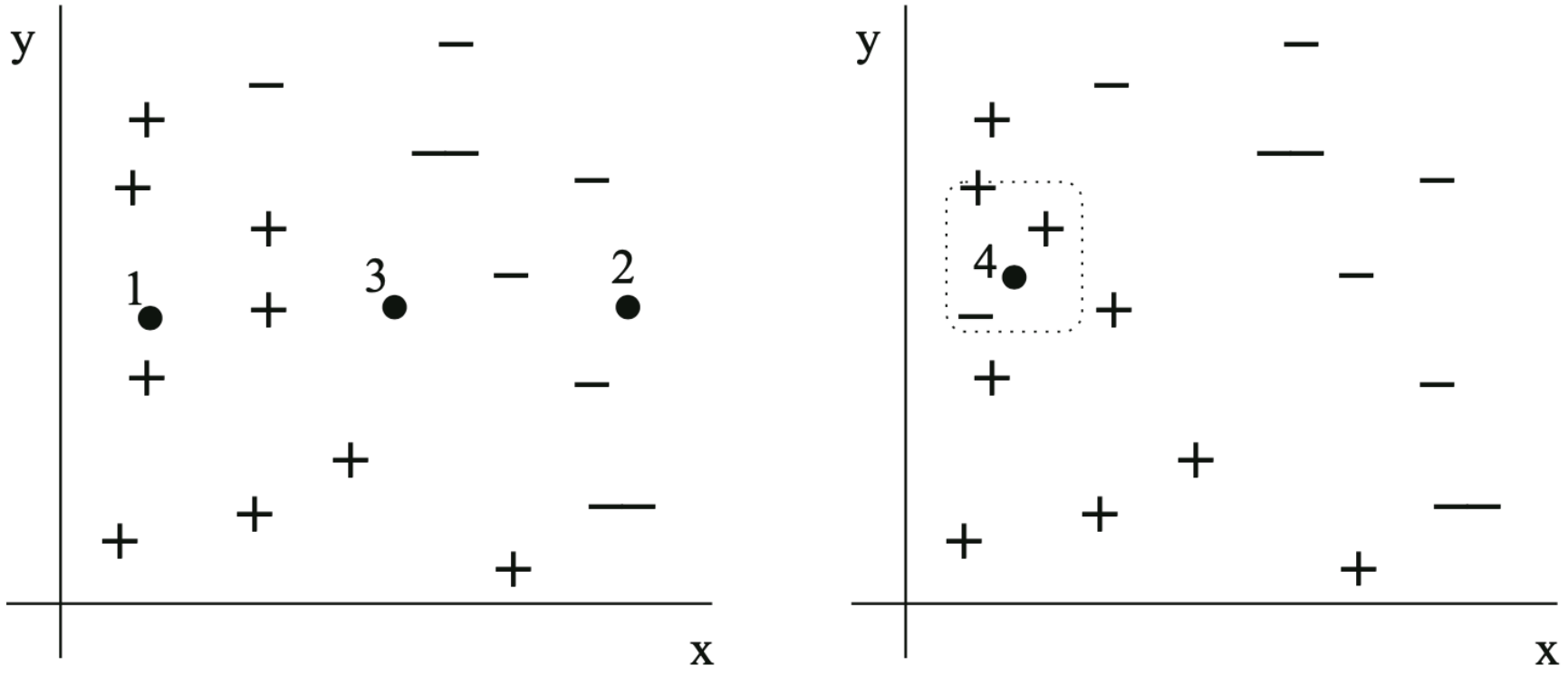
Example	Shape	Crust		Filling		Class	# differences
		Size	Shade	Size	Shade		
x	Square	Thick	Gray	Thin	White	?	–
ex_1	Circle	Thick	Gray	Thick	Dark	pos	3
ex_2	Circle	Thick	White	Thick	Dark	pos	4
ex_3	Triangle	Thick	Dark	Thick	Gray	pos	4
ex_4	Circle	Thin	White	Thin	Dark	pos	4
ex_5	Square	Thick	Dark	Thin	White	pos	1
ex_6	Circle	Thick	White	Thin	Dark	pos	3
ex_7	Circle	Thick	Gray	Thick	White	neg	2
ex_8	Square	Thick	White	Thick	Gray	neg	3
ex_9	Triangle	Thin	Gray	Thin	Dark	neg	3
ex_{10}	Circle	Thick	Dark	Thick	White	neg	3
ex_{11}	Square	Thick	White	Thick	Dark	neg	3
ex_{12}	Triangle	Thick	White	Thick	Gray	neg	4

(Kubat, pp. 44, 2017)

The k-Nearest-Neighbour Rule

- Idea:
  - one object can be misleading
  - **k most similar training objects** should guide us to more reliable decisions
- The simplest version of the k-NN classifier (Kubat, 2017)
  1. Among the training examples, identify the k nearest neighbours of **x** (examples most similar to **x**).
  2. Let c_i be the class most frequently found among these k nearest neighbours.
  3. Label **x** with c_i.
- *k* should be an odd number to avoid ties

k-NN classifier - an illustration



- Object 3 - a borderline case
- Object 4 - would be misclassified by a 1-NN classifier

Measuring similarity

- Objects as vectors:
  - $\mathbf{x} = \langle x_1, \ldots, x_n \rangle$
  - $\mathbf{y} = \langle y_1, \ldots, y_n \rangle$
- Euclidean distance

$$d_E(\mathbf{x}, \mathbf{y}) = \sqrt{\Sigma_{i=1}^{n}(x_i - y_i)^2}$$

- More general similarity measure

$$d_M(\mathbf{x}, \mathbf{y}) = \sqrt{\Sigma_{i=1}^{n} d(x_i, y_i)}$$

  - where d(…) - distance between attribute values, e.g.
    - for continuous, $d(x_i, y_i) = (x_i - y_i)^2$
    - for discrete, $d(x_i, y_i) = 1$, if $x_i = y_i$, 0 otherwise
    - binary values ⇨ Hamming distance

Distance vs similarity

1. The distance must never be negative;
2. The distance between two identical vectors, $\mathbf{x}$ and $\mathbf{y}$, is zero;
3. The distance from $\boldsymbol{x}$ to $\boldsymbol{y}$ is the same as the distance from $\boldsymbol{y}$ to $\boldsymbol{x}$;
4. The metric must satisfy the triangular inequality:
   $d(\boldsymbol{x}, \boldsymbol{y}) + d(\boldsymbol{y}, \boldsymbol{z}) \geqq d(\boldsymbol{x}, \boldsymbol{z})$.

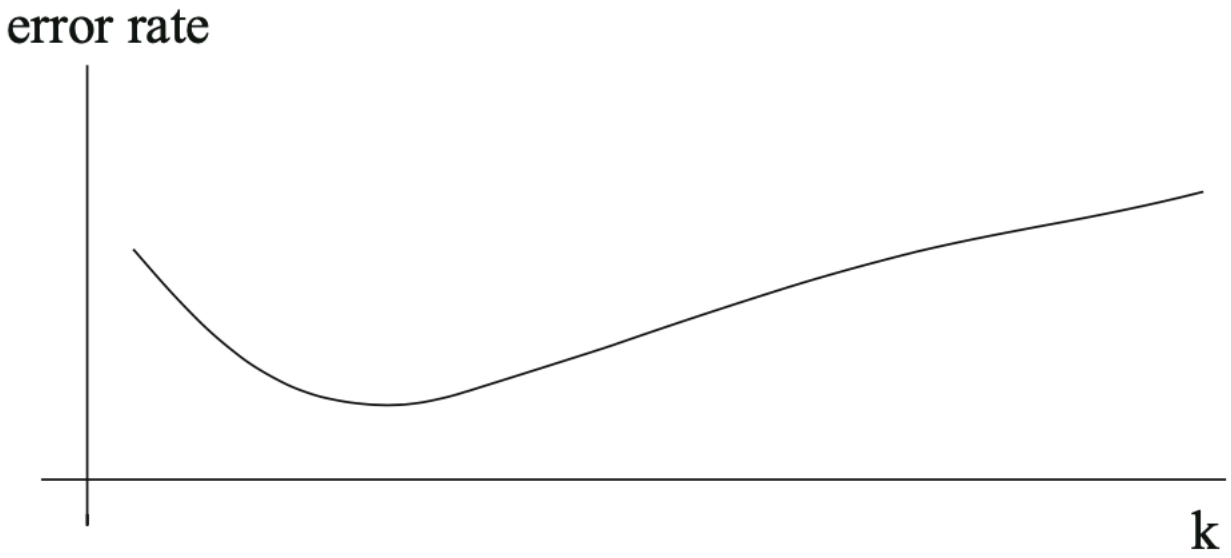
- Different distance measures:
  - the polar distance, the Minkowski metric, and the Mahalanobis distance.
- Not every similarity is distance and *vice versa*
- However, similarity may be sufficient for k-NN

Problems for k-NN

- Irrelevant attributes
  - smaller number of attributes — bigger problems
  - attribute selection — see the previous lecture
- Scales of attribute values
  - large domains vs small domains
    - larger values dominate the object distance (similarity)
  - e.g. $\boldsymbol{x} = \langle \text{true}, 0.2, 254 \rangle$ and $\boldsymbol{y} = \langle \text{false}, 0.1, 194 \rangle$
- Normalising attribute values
  - mapping onto the $[0, 1]$ range
  - $x = (x - \text{MIN}) / (\text{MAX} - \text{MIN})$
  - challenges:
    - distorted interpretation of attributes
    - binary attributes starts dominating
  - different approaches to normalisation

Problems for k-NN

- Increasing k has limits



- Curse of Dimensionality
  - growing number of attributes ⇒ growing computational cost
  - „as we increase the number of attributes, the number of training examples needed to fill the instance space with adequate density grows very fast, perhaps so fast as to render the nearest-neighbour paradigm impractical."

Weighted Nearest Neighbours

- Different neighbours may be located in different distances (similarities)
- Idea: give more credit to closer (more similar)
- Scheme
  - weight of each of the nearest neighbours is made proportional to its distance from $\boldsymbol{x}$
    - the closer the neighbour, the greater its impact.
  - weighted voting: sums of weights for different classes
  - normalised weights

$$w_i = \begin{cases} \frac{d_k - d_i}{d_k - d_1}, & d_k \neq d_1 \\ 1 & d_k = d_1 \end{cases}$$

  - where $d_1, \ldots, d_k$ - distances
  - only k-1 neighbours are taken into account

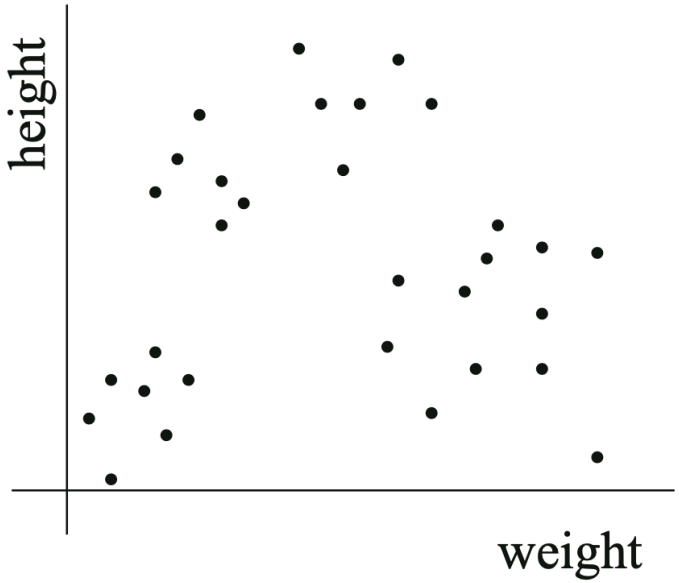
UNSUPERVISED LEARNING

Learning knowledge directly from data

- Supervised learning — induction from pre-classified examples (training samples)
- Unsupervised learning
  - discovering useful properties of available data
  - in the vast majority of cases: searching for groups (called clusters) of similar examples
- Questions:
  - How to group examples (data samples) together?
  - How to measure similarity of examples?
  - How to find groups of similar examples?
  - How to represent groups or to identify what is common for a group?
  - How to use the knowledge about the structure of clusters?
- Applications
  - e.g. „centres for Bayesian or RBF (Radial Basis Function) classifiers, as predictors of unknown attribute values, and even as visualisation tools for multidimensional data" (Kubat, 2017, pp. 273)

Cluster Analysis – Tasks

- Identifying Groups of Similar Examples
- Representing Clusters, e.g.
  - centroid vector — **centroids**
    - average of attributes of objects belonging to a given cluster
    - discrete attributes must be turned into numerical ones



- Cluster properties
  - non-overlapping (strict clustering vs fuzzy clustering)
  - internal density and consistency
    - „examples should be relatively close to each other, certainly much closer than to the examples from the other clusters”
  - the number and balance of clusters
- Measuring distances
  - to clusters and between clusters
- Assignment of an example to a cluster

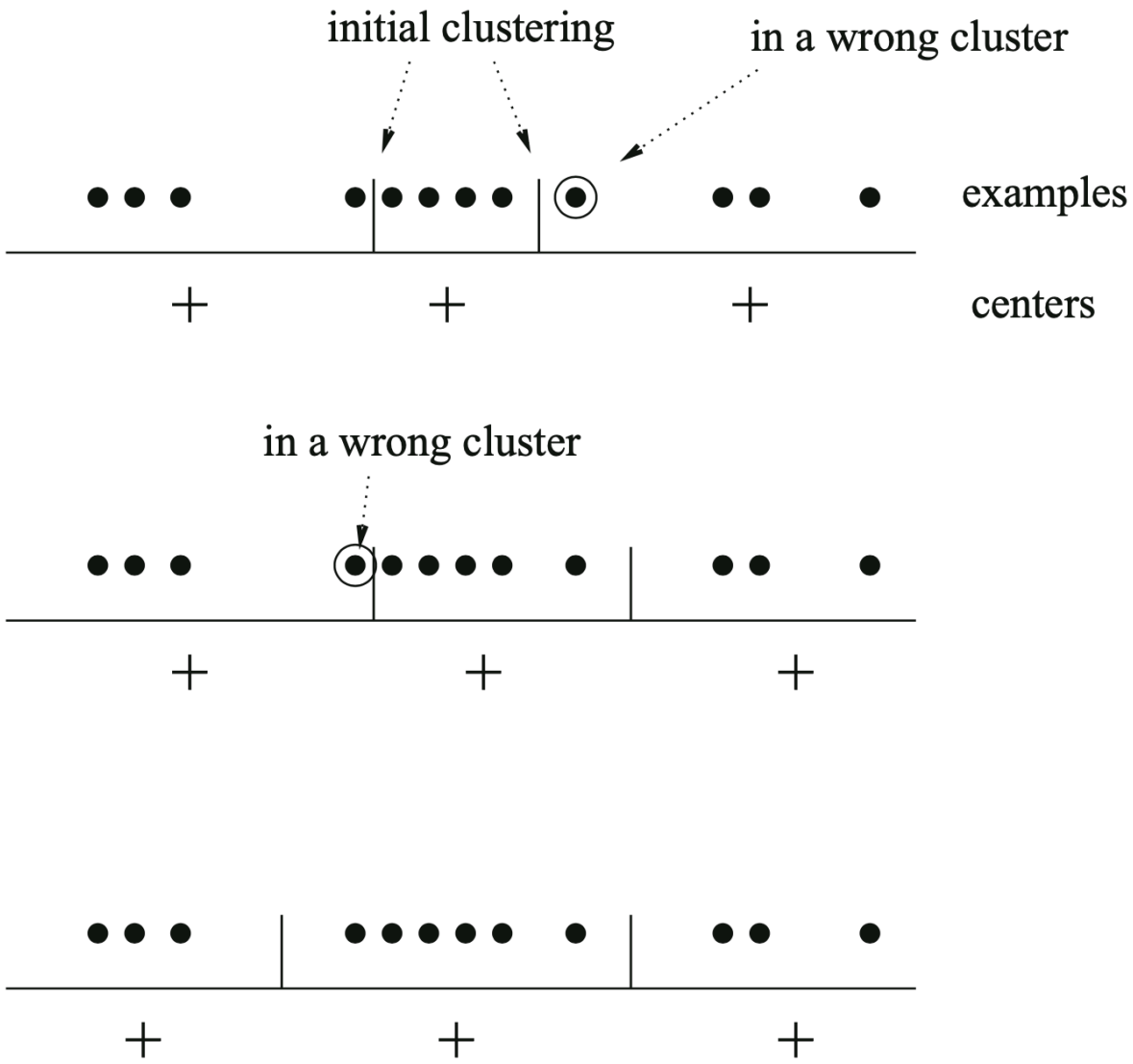
k-Means

- Input:
  a set of examples without class labels, user-set constant k

1. Create k initial clusters. For each, calculate the coordinates of its centroid, C_i, as the numeric averages of the attribute values in the examples it contains.
2. Choose an example, $\boldsymbol{x}$, and find its distances from all centroids. Let j be the index of the nearest centroid.
3. If $\boldsymbol{x}$ already finds itself in the j-th cluster, do nothing. Otherwise, move $\boldsymbol{x}$ from its current cluster to the j-th cluster and recalculate the centroids.
4. Unless a stopping criterion has been satisfied, repeat the last two steps for another example.

- Stoping criterion:
  each training example already finds itself in the nearest cluster.

k-Means - example



(Kubat, 2017, pp. 278)

k-Means - problems

- Attributes of different domains ⇒ normalisation
  - issue familiar from k-NN
- Computational Aspects of Initialisation
  - affects the number of objects transfers
- Initialisation
  - random
  - some background knowledge, e.g. preference to some attributes
  - the final cluster composition may depend on the initialisation

Expansions to k-Means

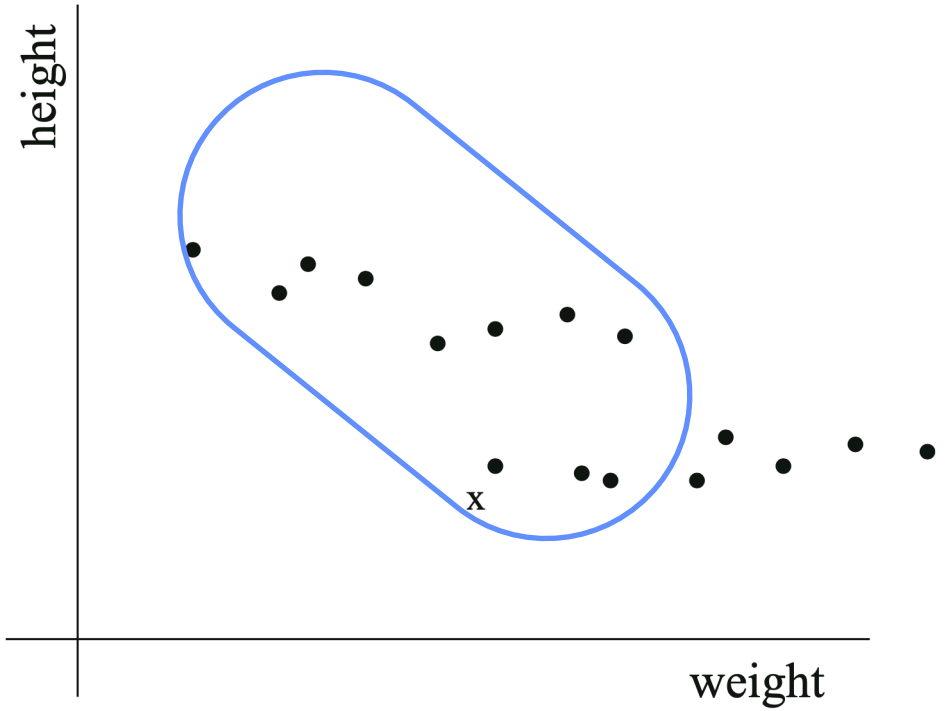
- Quality of the Clusters
  - supervised measures (external evaluation) — comparison to a known classification of objects
    - e.g. entropy of classes over clusters, purity of clusters, IRad, etc.
  - unsupervised measures (internal evaluation) — properties of clusters against the similarity (distance measure)
    - SD (summed distances) — summed distances of all examples from their clusters' centroids

$$SD = \sum_{j=1}^{K} \sum_{i=1}^{n_j} d(\mathbf{x}_i^{(j)}, \mathbf{c}_j)$$

    - Dunn index, Silhouette coefficient
- Optimisation
  - alternative initialisations: different initial code vectors ⇒ k-Means ⇒ evaluation
  - different k - values
  - post-processing: merging or splitting clusters
- Hierarchical applications: top-down, increasing k

Hierarchical clustering

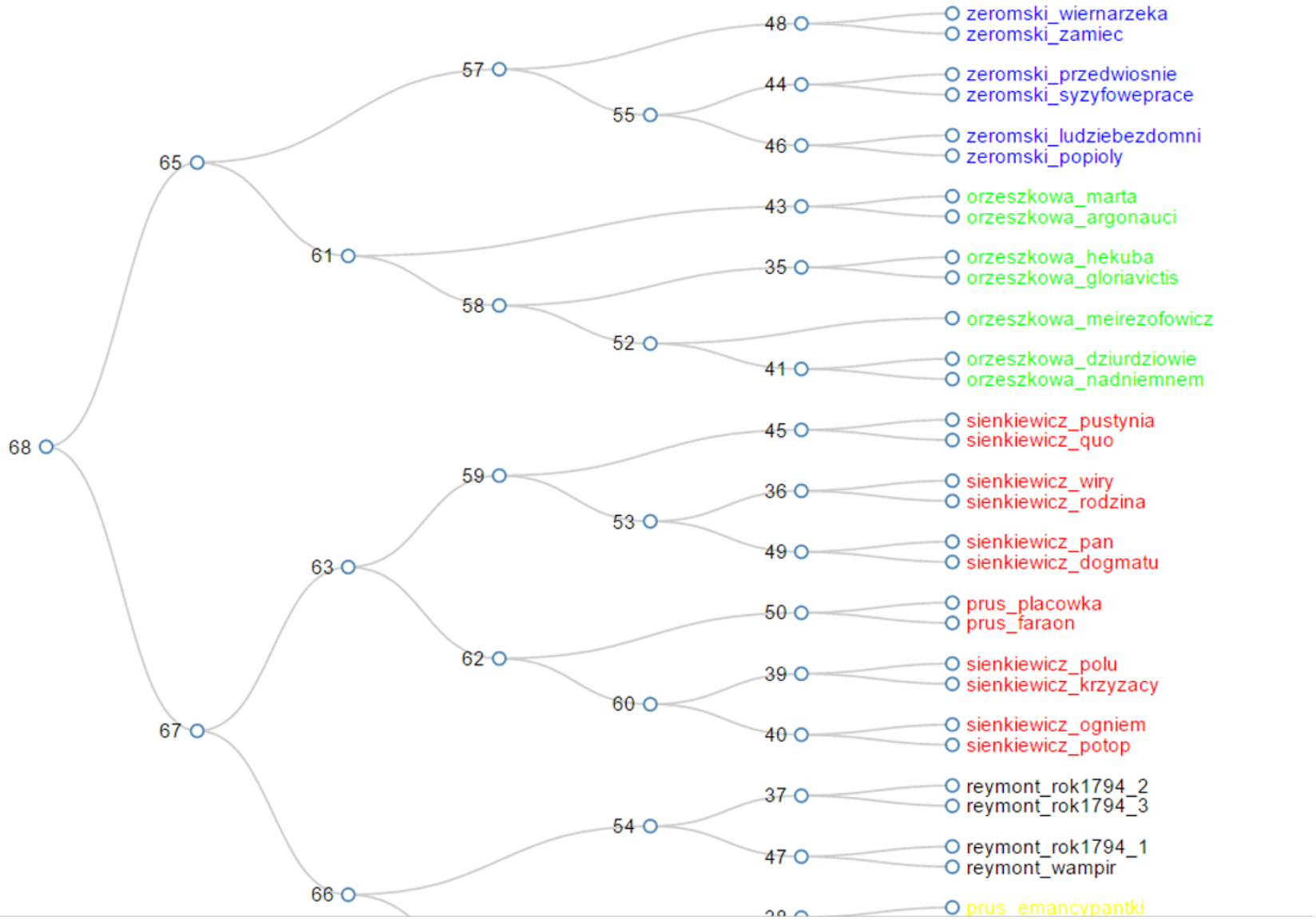
- Schemes
  - top-down
    - coarse-grained clustering (larger clusters) ⇒ in-cluster, finer grained
    - e.g. hierarchical k-Means
  - bottom-up
    - small clusters ⇒ clustering clusters
- No need for defining the k value
- k-Means may misinterpret the cluster structure



Hierarchical clustering

- Distance to cluster
  - cluster as a set: distances between all pairs of examples, $[\boldsymbol{x}, \boldsymbol{y}]$
  - single-link — minimum from the distances
    - very thigh cluster of high internal similarity
    - fine grained clustering
  - average-link — average from the distances
  - max-link — maximum from the distances
    - loosely connected clusters
    - coarse-grained clustering
- Advantage:
  - the number of clusters does not need to be pre-defined

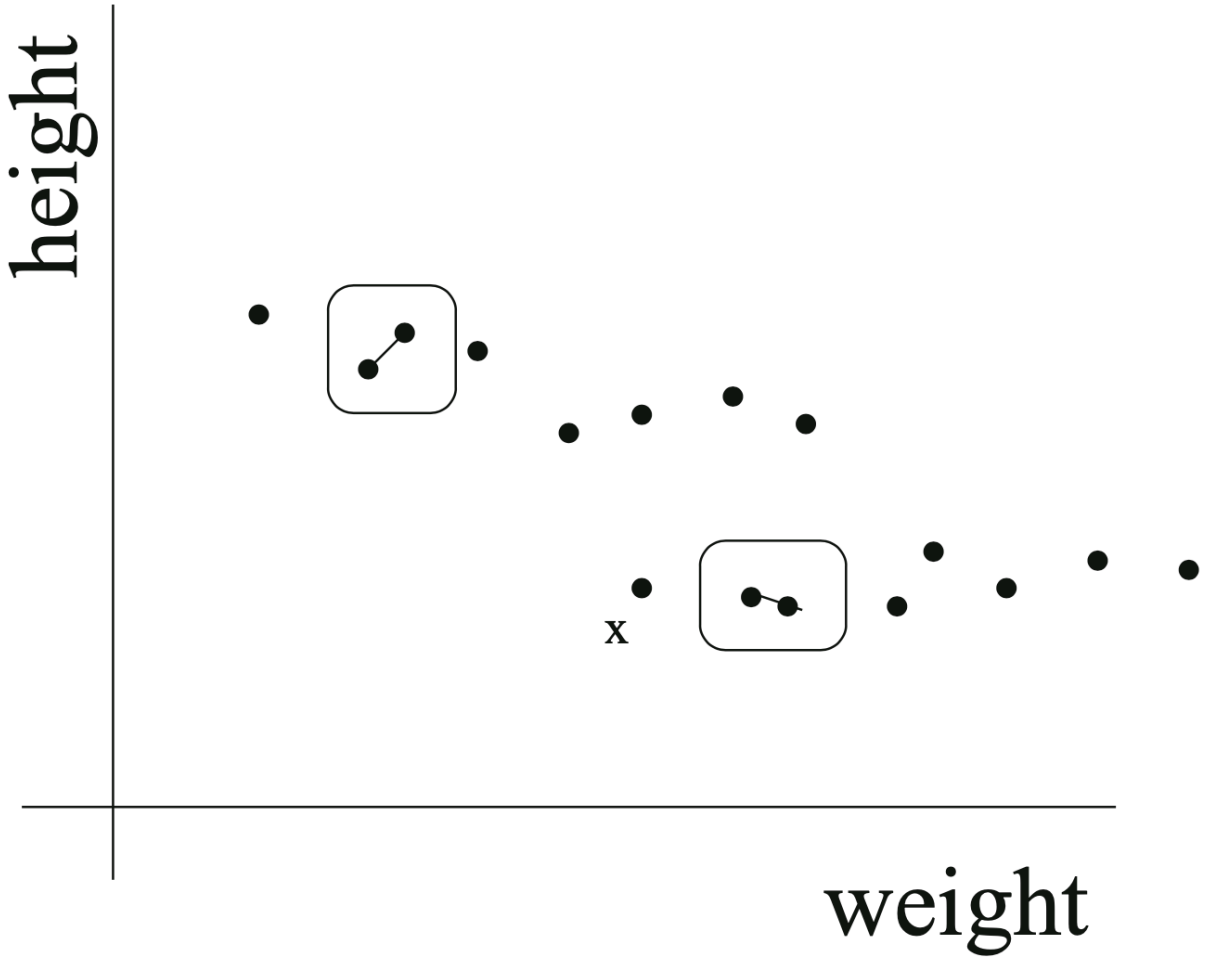
WebSty: Visualisation http://ws.clarin-pl.eu/websty.shtml?en



Hierarchical aggregation

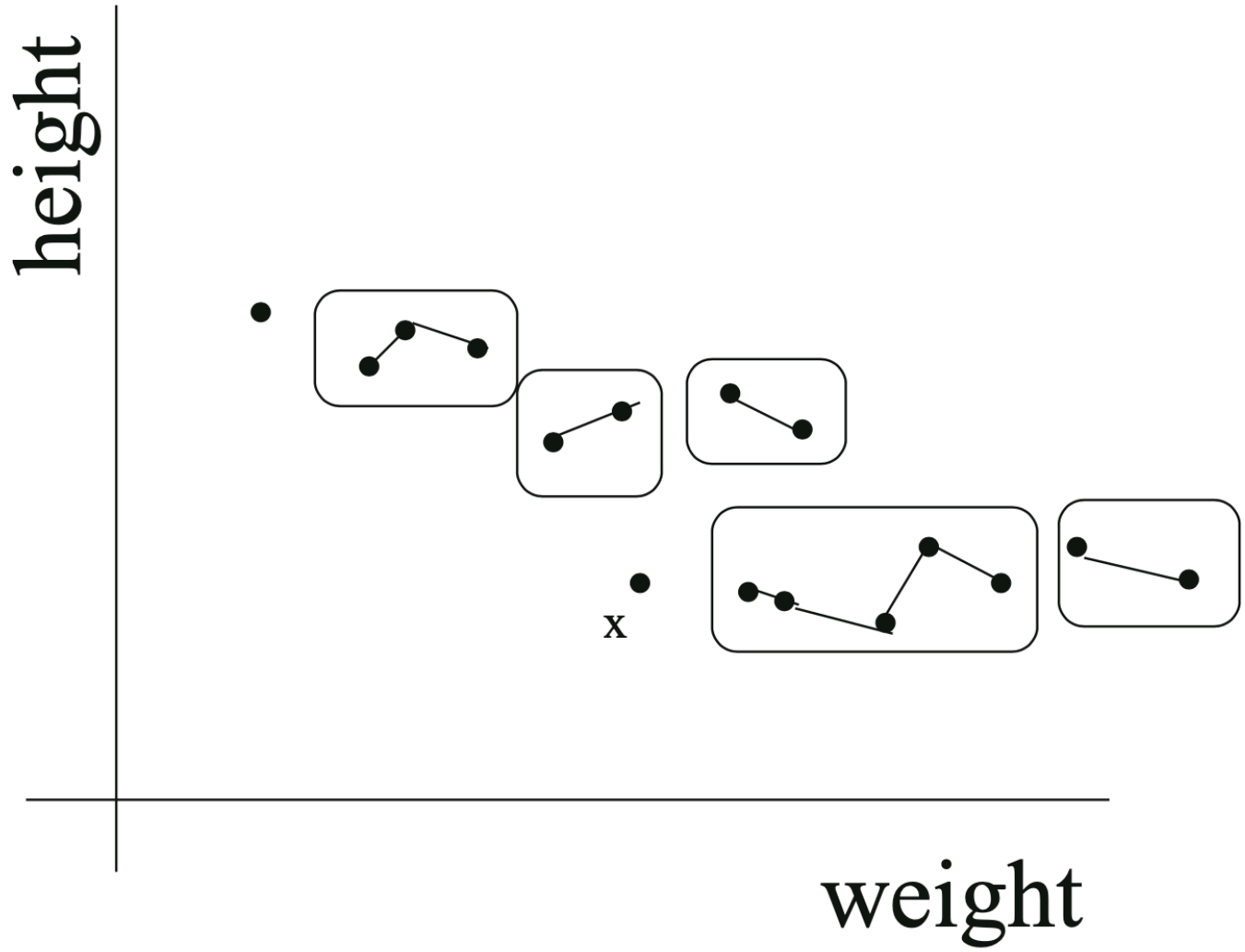
- Input: a set of examples without labels
- Let each example form one cluster. For N examples, this means creating N clusters, each containing a single example.
- Find a pair of clusters with the smallest cluster-to-cluster distance.
- Merge the two clusters into one, thus reducing the total number of clusters to N-1.
- Unless a termination criterion is satisfied, repeat the previous step.

Hierarchical aggregation



(Kubat, 2017, pp 286)

Hierarchical aggregation



(Kubat, 2017, pp 286)

Machine Learning Frameworks

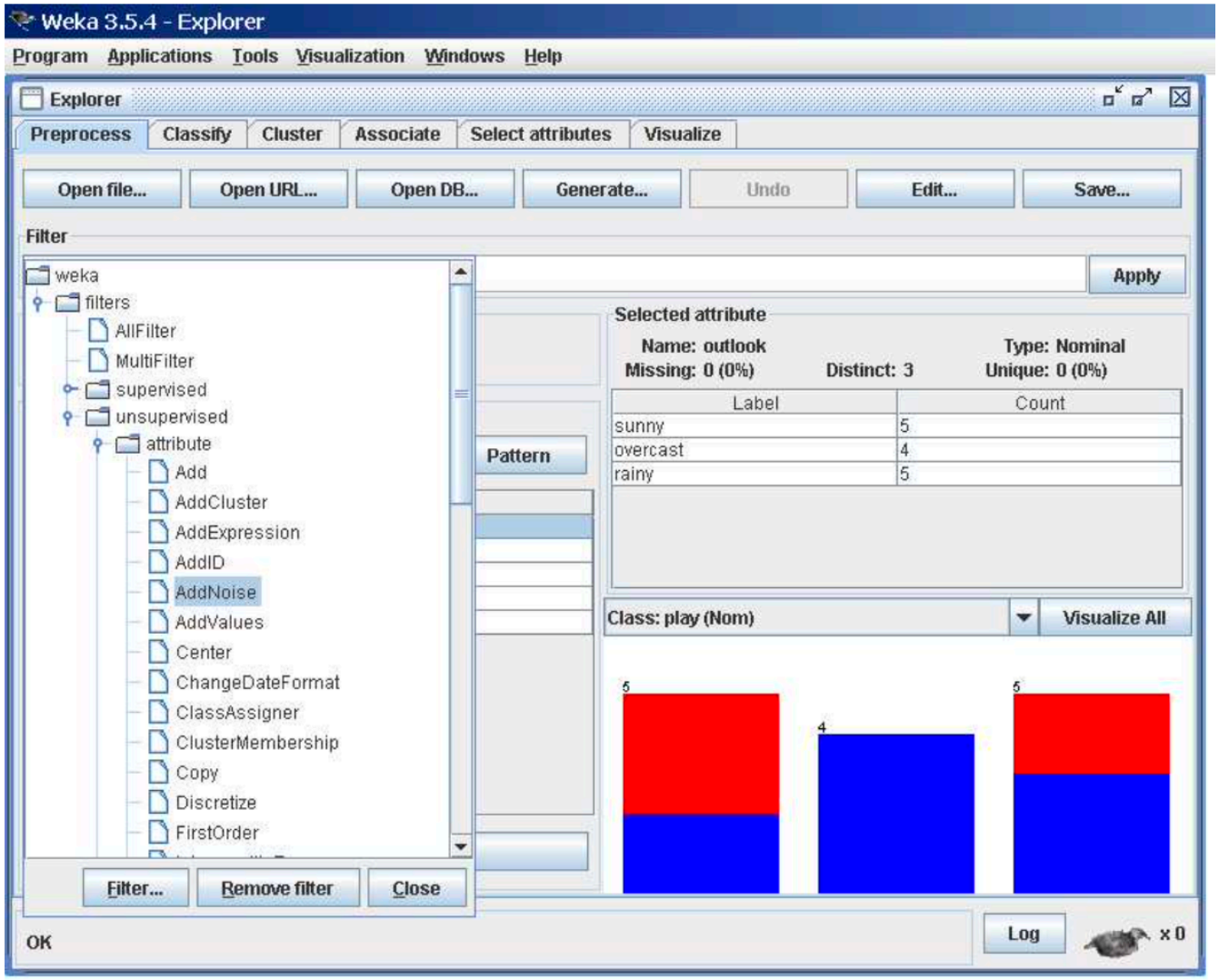
- ML frameworks
  - programming libraries, components,
  - tools, applications, GUI
  - planning processing pipelines
  - analytical and visualisation tools
- Examples
  - WEKA - versatile system for data analysis and ML
  - TiMBL - Tilburg Memory-Based Learner
  - scikit-learn - Python-based framework for Machine Learning
  - CLUTO - data clustering

WEKA - a Data Mining system

- Main features
  - integrated data analysis, data mining and Machine Learning
  - written in Java
    - ready to use modules (binary)
    - but also access via Java libraries and classes
  - user interface
    - command line (SimpleCLI)
    - Graphical User Interface
  - several dozens of tools, main algorithms for features selection, data clustering and ML supported
- Main modules
  - Explorer
  - Experimenter
  - KnowledgeFlow

WEKA - Explorer

- Preprocess
- Classify
- Cluster
- Associate
- Select attributes
- Visualise



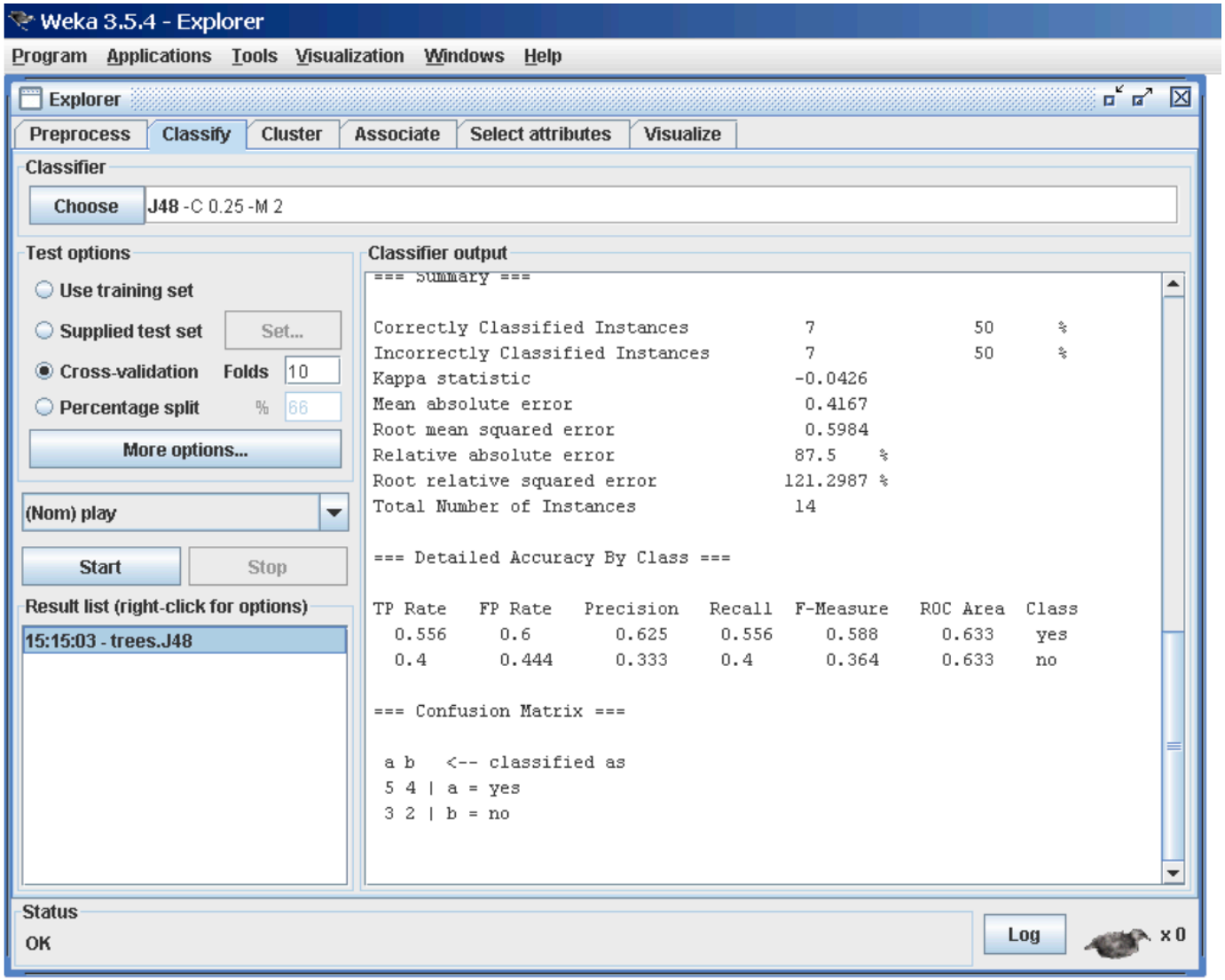
WEKA – ARFF format

- ARFF = Attribute-Relation File Format
- Structure: **Header** followed by **Data**
- Data types: numeric, string, date, relational (linking between sets)

```
% 1. Title: Iris Plants Database %
% 2. Sources:
        % (a) Creator: R.A. Fisher
        % (b) Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
        % (c) Date: July, 1988
@RELATION iris
@ATTRIBUTE sepallength.    NUMERIC
@ATTRIBUTE sepalwidth      NUMERIC
@ATTRIBUTE petallength     NUMERIC
@ATTRIBUTE petalwidth      NUMERIC
@ATTRIBUTE class           {Iris-setosa, Iris-versicolor, Iris-virginica}
@DATA
5.1,3.5,1.4,0.2,Iris-setosa
4.9,3.0,1.4,0.2,Iris-setosa
4.7,3.2,1.3,0.2,Iris-setosa
```

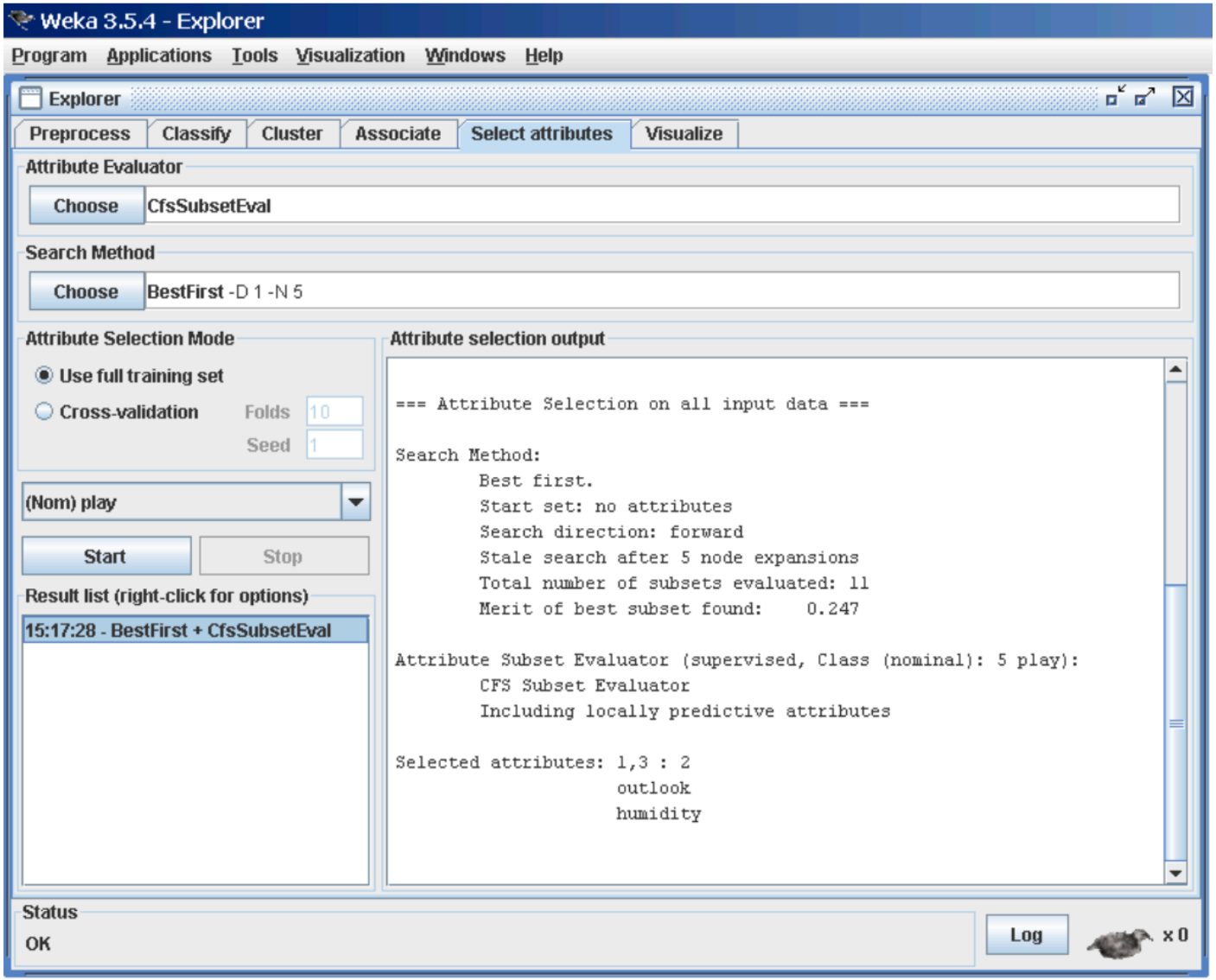
WEKA – Explorer – Classification

1. Selecting a Classifier
2. Test Options
   - training set
   - test set
   - cross-validation
   - split
3. The Class Attribute
4. Training a Classifier
5. The Classifier Output Text
6. The Result List

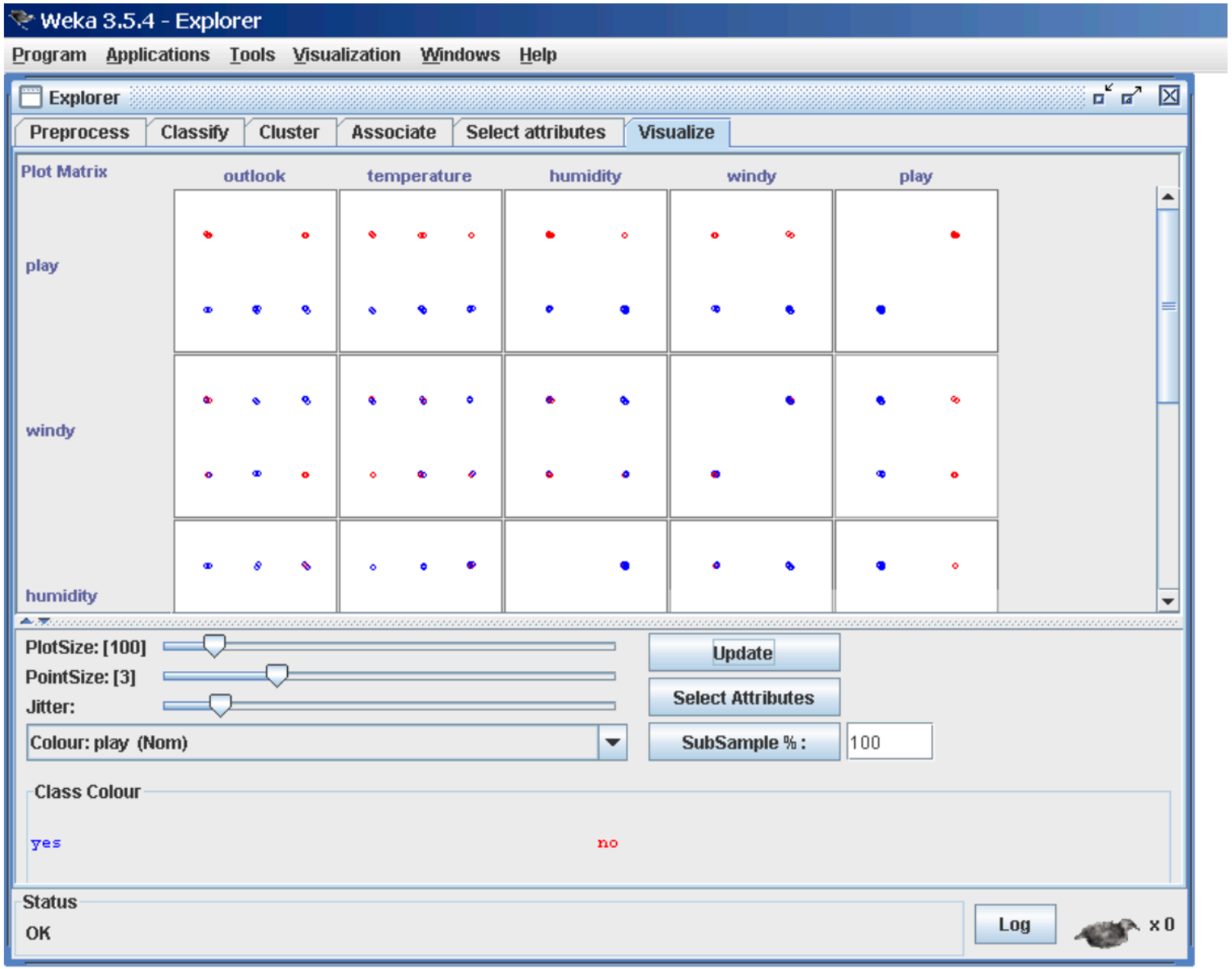


WEKA – Explorer – Selecting Attributes

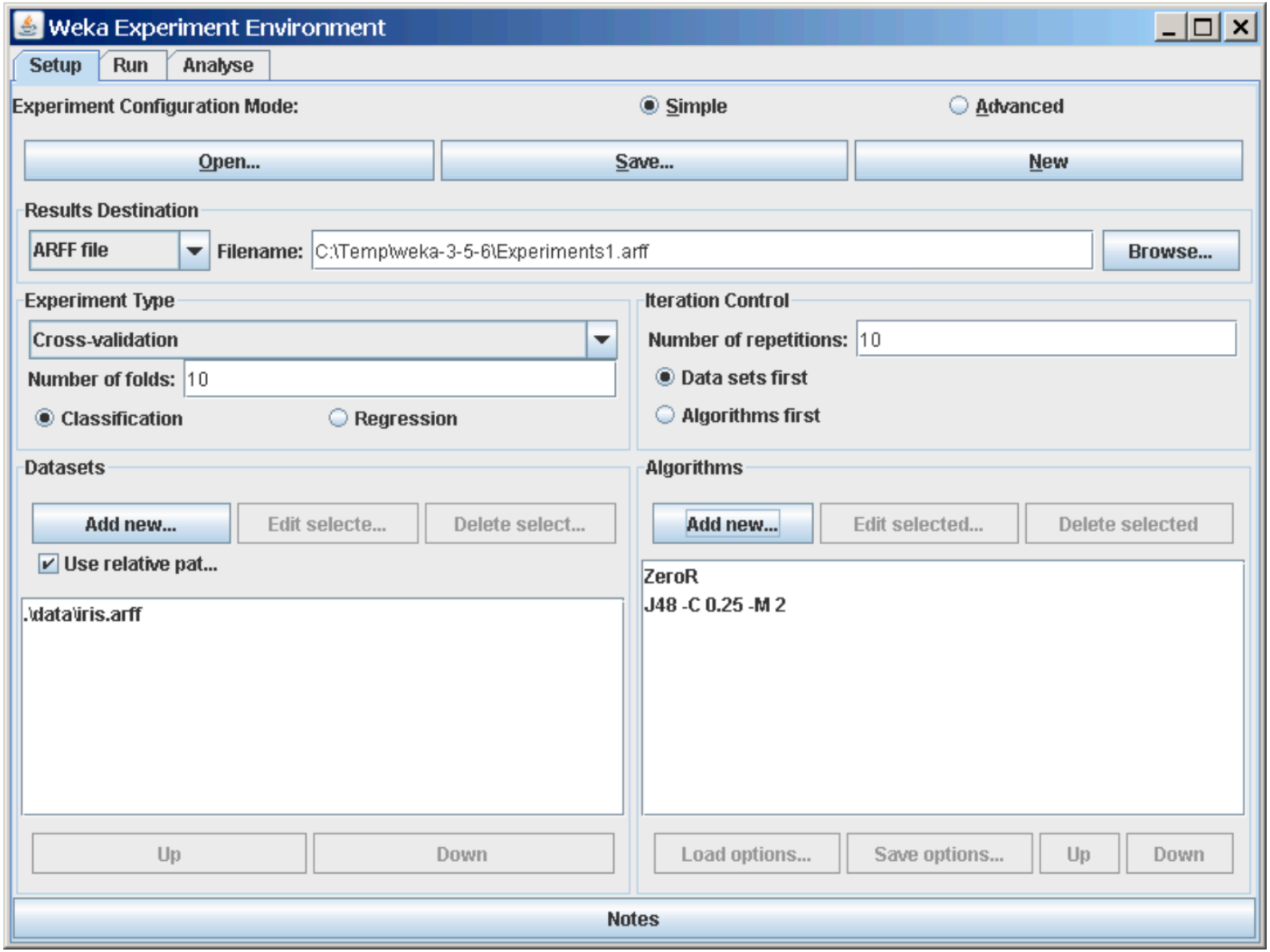
- Searching and Evaluating
  - full training set.
  - cross-validation
- Performing Selection



WEKA – Explorer – Visualizing

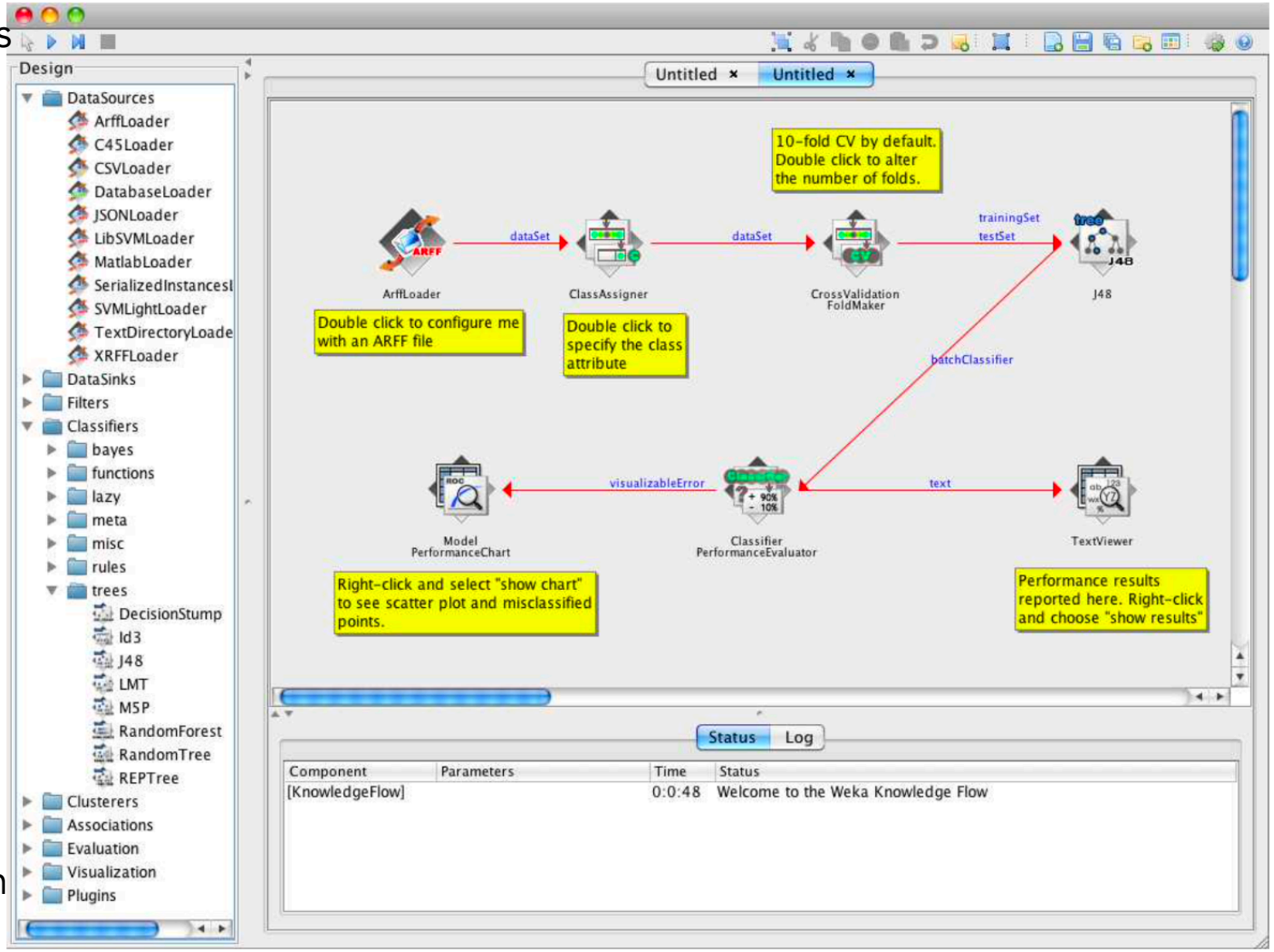


WEKA – Experimenter



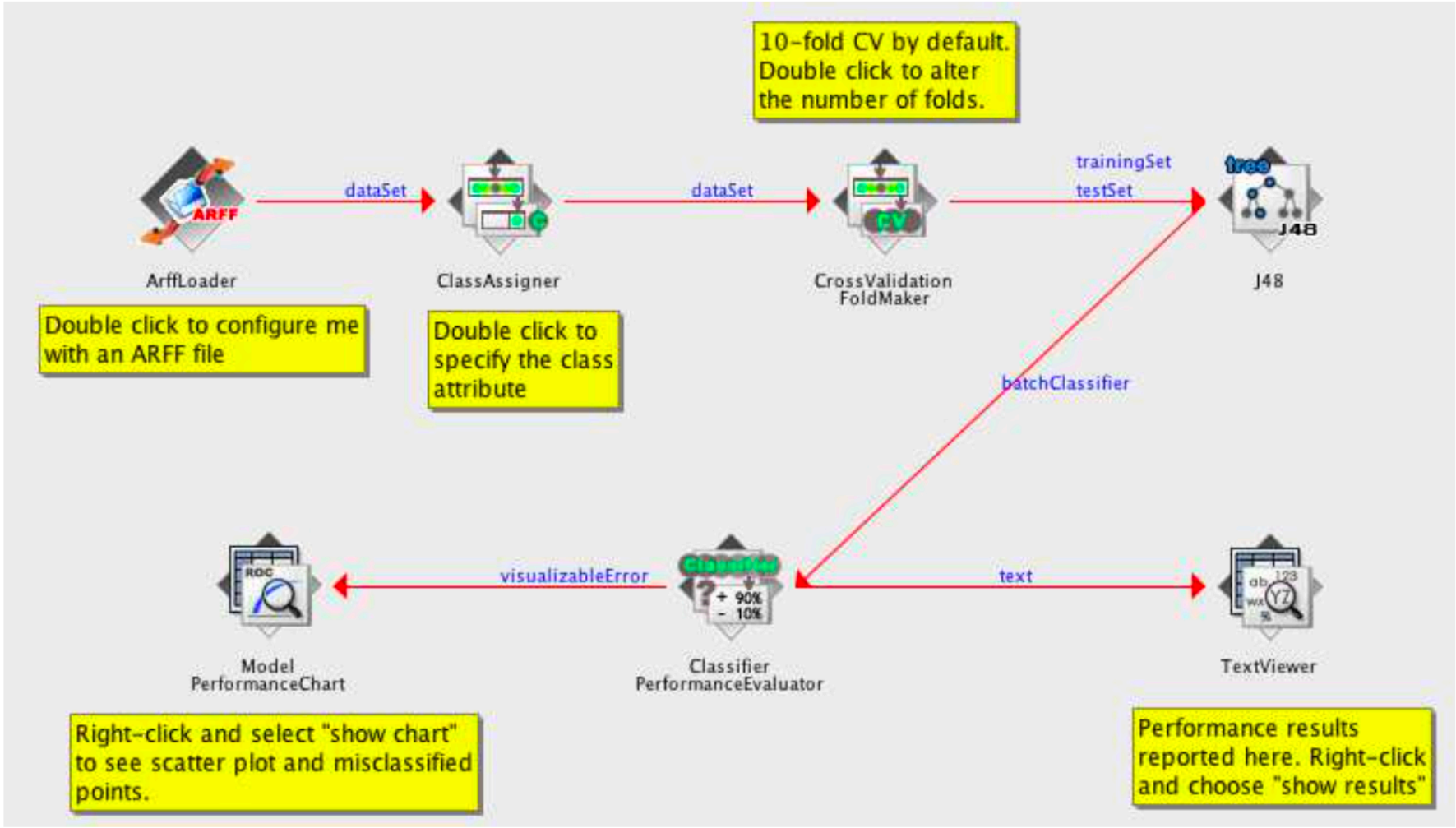
WEKA – KnowledgeFlow

- DataSources
- DataSinks
- Filters
- Classifiers
- Clusterers
- Evaluation
- Training Set Maker
- Cross Validation Fold Maker
- Class Assigner
- Instance Stream To Batch Maker
- …
- Visualization



WEKA – KnowledgeFlow – an example

Loading an ARFF file (batch mode) and performing a cross-validation using J48



scikit-learn – Python framework for ML

- Python library
  - used on the level of the Python source code
- Main modules
  - Preprocessing — feature extraction and normalisation
  - Dimensionality reduction — algorithms for mapping between vector spaces, e.g.
    - Principal component analysis (PCA)
    - a variant of Singular Value Decomposition (SVD)
    - Latent Dirichlet Allocation (LDA) — famous from topic modelling for texts
  - Classification — more than 100 classifiers and versions
  - Regression — mostly the same algorithms like in Classification
  - Model selection and evaluation
    - Cross-validation
    - Tuning the hyper-parameters
    - Metrics and scoring: quantifying the quality of predictions
    - Validation curves: plotting scores to evaluate models
  - Clustering — e.g., k-Means, spectral clustering, mean-shift,

scikit-learn – Example

- Pipelines: chaining pre-processors and estimators

```
>>> from sklearn.preprocessing import StandardScaler
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.pipeline import make_pipeline
>>> from sklearn.datasets import load_iris
>>> from sklearn.model_selection import train_test_split
>>> from sklearn.metrics import accuracy_score
...
>>> # create a pipeline object
>>> pipe = make_pipeline(
...     StandardScaler(),
...     LogisticRegression()
... )
...
>>> # load the iris dataset and split it into train and test sets
>>> X, y = load_iris(return_X_y=True)
>>> X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
...
>>> # fit the whole pipeline
>>> pipe.fit(X_train, y_train)
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('logisticregression', LogisticRegression())])
>>> # we can now use it like any other estimator
>>> accuracy_score(pipe.predict(X_test), y_test)
0.97...
```

Literaturę

- Miroslav Kubat. (2017) An Introduction to Machine Learning. Springer. (available via WUST network)
- https://en.wikipedia.org/wiki/Cluster_analysis#Evaluation_and_assessment
- **scikit-learn** - Machine Learning in Python. https://scikit-learn.org/stable/
- **TiMBL** - Tilburg Memory-Based Learner. http://languagemachines.github.io/timbl/
- Remco R. Bouckaert, Eibe Frank, Mark Hall, Richard Kirkby, Peter Reutemann, Alex Seewald and David Scuse. (2013) **WEKA** Manual for Version 3-7-8. https://wekatutorial.com/doc/WekaManual-3-7-8.pdf
- **CLUTO** - Software for Clustering High-Dimensional Datasets. Karypis Lab. http://glaros.dtc.umn.edu/gkhome/cluto/cluto/overview